

COURSE 2 : BASICS LARAVEL

Dr. Sourour BELHAJ RHOUMA



COURSE OUTCOMES

- ✓ Match each given URL to a specific process using routing
 - ✓ Group related processes using controllers
 - ✓ Retrieve data from an HTTP request using Request
 - ✓ Return content in text, HTML, JSON, etc. formats using Response
 - ✓ Integrate data into templates using Blade
 - ✓ Interact with the user using forms
 - ✓ Create, update, and track changes to a database system using migrations
 - ✓ Facilitate communication with a database using Eloquent
-

SUMMARY

1. Preliminaries
 - 2. Routing**
 - 3. Controllers**
 - 4. Requests**
 - 5. Responses**
 6. Views
 7. Database
-

ROUTING

- Overview
- Definition
- Basic Routing
- Named Routing
- URL Generation and Redirection

ROUTING: OVERVIEW

- Routing = matching a given URL to a specific process
- Benefit: having attractive URLs for good SEO and visitor comfort
- Ex: `read/intro-to-laravel` instead of `index.php?article_id=57`
- Defining a route = URL path pattern + process

ROUTING: DEFINITION

Laravel's routing system allows you to define application URLs and bind them to the appropriate controllers.

Routes are defined in different files depending on their use:

- ✓ ***web.php***: For standard web routes (without a prefix). (For example, /, /about, /contact)
 - ✓ ***api.php***: For API routes. (Always prefixed with /api, automatically.)
Authentication is also handled differently. Via a token system with Laravel Sanctum)
 - ✓ ***console.php***: For console commands. We won't cover these in class, but they're an important element of Laravel's flexibility..
-

ROUTING: DEFINITION

- **Location:**

File « routes/web.php »

- **Syntaxe:**

`Route::methodHTTP('path',closure);`

- **Example:**

```
Route::get('/', function(){  
    return view('welcome');  
})
```

AVAILABLE ROUTER METHODS

The router allows you to register routes that respond to any HTTP verb:

```
Route::get($uri, $callback);  
Route::post($uri, $callback);  
Route::put($uri, $callback);  
Route::patch($uri, $callback);  
Route::delete($uri, $callback);  
Route::options($uri, $callback);
```

BASIC ROUTING

The most basic Laravel routes accept a URI and a closure, providing a very simple and expressive method for defining routes and behavior without complicated routing configuration files:

```
use Illuminate\Support\Facades\Route;
Route::get('/greeting', function () {
    return 'Hello World';
});
```

ROUTING:ROUTE PARAMETER

- **Problem:** How to send data via the path.
- **Solution:** Use variable segments in the path.
- **Append syntax:**
- Add {parameterName} to the path.
- Pass \$parameterName as a parameter to the callback function.
- **Example:**

```
Route::get('/hello/{prenom}', function ($prenom) {  
    return 'Hello ' . $prenom;  
});
```

- **Testing paths:**

/hello/VotrePrenom

/hello

ROUTING: ROUTE PARAMETER

Sometimes you'll need to capture segments of the URL in your route. For example, you might need to capture a user's ID from the URL. You can do this by setting route parameters:

```
Route::get('/user/{id}', function ($id) {  
    return 'User '.$id;  
});
```

ROUTING: OPTIONAL ROUTE PARAMETER

Syntax for adding optional parameters

1. Apply {parameterName?} to path: Makes the parameter optional.
2. Pass \$parameterName = defaultValue: Sets a default value.

Example:

```
Route::get('/hello/{prenom?}', function ($prenom = 'world')  
{  
    return 'Hello ' . $prenom;  
});
```



Testing paths :

- /hello/VotrePrenom
 - /hello
-

ROUTING: OPTIONAL PARAMETER ROUTE

You may need to specify a route parameter that is not always present in the URI. You can do this by placing a ? mark after the parameter name. Make sure to give a default value to the corresponding route variable:

```
Route::get('/user/{name?}', function ($name = null) {  
    return $name;  
});
```

```
Route::get('/user/{name?}', function ($name = 'John') {  
    return $name;  
});
```

NAMED ROUTING

Named routes allow for the convenient generation of URLs or redirects for specific routes.

To name a route, simply chain the name method to the route definition:

```
Route::get('/hello/{prenom?}', function ($prenom = 'world') {  
    return 'Hello ' . $prenom;  
})->name('hello');
```

ROUTING: URL GENERATION AND REDIRECTION

URL Generation

- Allows you to avoid hardcoding href values in templates.
- Makes it easier to modify URLs if they change.

URL Generation Syntaxes

- `$url = route('nameOfRoute');`
- With parameter(s): `route('nameOfRoute', ['param' => value, ...]);`

Redirection Syntaxes

- `redirect()->route('nameOfRoute');`
 - With parameter(s): `redirect()->route('nameOfRoute', ['param' => value, ...]);`
-

ROUTING: URL GENERATION AND REDIRECTION

Example

1. The route named hello has the path `/hello/{prenom?}`.
2. In the route with path `/`, replace `return view( 'welcome' )` with:
`redirect()->route('hello', ['prenom' => 'Redirected User']);`
3. Test the `/` path to see the redirect in action

CONTROLERS

- Presentation
- Definition
- example

CONTROLLERS: DEFINITION

- **Location:** Controllers are located in `app/Http/Controllers`.
- **Creation Command:** To create a controller, use the following command from the project root:

```
php artisan make:controller NameController
```

- **Route syntax:**

Using a controller in a route:

```
Route::methodHTTP('path', [NameController::class, 'nameAction']);
```

Or

```
Route::methodHTTP('path', 'NameController@nameAction');
```

CONTRRLERS: PRESENTATION

- Controller: A class that groups methods that act on the same resource.
 - Action: A function or method that receives a request, processes it, and returns a response (text, HTML, XML, JSON, image, redirection, 404 error, etc.).
 - Inheritance: To simplify certain processing, the controller can inherit from the Controller class.
-

CONTROLLERS: EXAMPLE

- **Steps to Create and Use a Controller**

1. Create a controller with the command:

```
php artisan make:controller LearnerController
```

2. Add the following methods to the class *LearnerController*:

```
public function index() {  
    return redirect()->route('hello', ['prenom' => 'Redirected User']);  
}  
  
public function hello($prenom = 'world') {  
    return 'Hello ' . $prenom;  
}
```

CONTROLLERS: EXAMPLE

- **Steps to Create and Use a Controller**

3. Replace routes with :

```
Route::get('/', [LearnerController::class, 'index']);
```

```
Route::get('/hello/{prenom?}', [LearnerController::class, 'hello'])->name('hello');
```

Or

```
Route::get('/', 'LearnerController@index');
```

```
Route::get('/hello/{prenom?}', 'LearnerController@hello')->name('hello');
```

4. Test routes to check the operation

REQUESTS

1. Access to the request
2. Retrieve request data

REQUESTS: ACCESS TO THE REQUEST

- An HTTP request often contains data (properties of a resource to be added/modified).
 - The server needs access to this data to perform processing.
 - To access the request:
 - ✓ Pass a Request instance as a function parameter.
 - ✓ **Syntax:** `functionName(Request $request)`
 - ✓ Any other parameters must be placed after the Request instance.
 - **Note:** Don't forget to import the Request class:
`use Illuminate\Http\Request;`
-

REQUESTS: RETRIEVE REQUEST DATA

Three ways to send data:

1. Via la query string (?param=valeur)
2. Via the path (route parameters)
3. Via query body (body)

Recovery Methods:

- ✓ `$request->all()`: All data (associative array)
 - ✓ `$request->input()`: All data or specific data
 - ✓ `$request->input('data_name')`: specific data
 - ✓ `$request->query('param')`: Data of query string
 - ✓ `$request->query()`: All the query string
 - ✓ `$request->data_name`: Direct access(body, query string or path)
 - ✓ `$request->input('objet.champ')`: For JSON data
-

RÉSPONSES

Response Types in Laravel

1. Returning Plain Text
 3. Returning a Custom Response Instance
 4. Returning a View
 5. Performing a Redirect
 - To a Controller Action
 - To a Relative Path
-

RESPONSES: RETURN TEXT

```
return 'returned text';
```

➔ Returns text in text/html format

```
return 'Welcome to our application !';
```

- Autoresponse format: text/html
 - Default HTTP status: 200 (OK)
 - Typical usage:
 - Simple messages
 - Quick tests
 - Minimalistic API responses
 - **Note:** To force a different content type, use `response()->text()`
-

RESPONSES: CUSTOM RESPONSE INSTANCE

```
return response($content, $statusCode)
```

```
-> header('NameHeader1', $vaue)
```

```
-> header('NameHearder2', $value)
```

➔ returns a Response instance with content, code and header.

```
return response(  
    'Customized content', // Content  
    201                  // HTTP code  
)->header('Content-Type', 'text/plain')  
->header('X-Custom-Header', 'Value');
```

RESPONSES: CUSTOM RESPONSE INSTANCE

Use Case:

- APIs requiring specific headers
- Precise control over HTTP code
- Non-standard formats (XML, plain text, etc.)

Advantages:

- Complete control over the response
 - Compatible with middleware
-

RÉSPONSES: RETURN A VIEW

Return view('nomVue')

➔ Return a view

```
return view('profile', [  
    'user' => User::find(1)  
]);
```

Best practices:

- Always pass data through an associative array
- Use **compact()** for simplicity:

```
$user = User::find(1);  
return view('profile', compact('user'));
```

RESPONSES: PERFORM REDIRECTS

Towards a controller action:

```
return redirect()->action([NameController::class, 'NameAction'],  
    tabAssParam);
```

➔ Redirects to an action with possible parameters

```
return redirect()->action(  
    [UserController::class, 'profile'],  
    ['id' => 1] // Parameters  
);
```

RÉSPONSES: PERFORM REDIRECTS

To a relative path:

```
return redirect('chemin/de/la/page.html');
```

➔ Redirects to a page

```
return redirect('users/profile'); // Relative path
```

```
return redirect('/absolute/path'); // Absolute path
```

Special Redirects:

`back()`: Return to the previous page

`route()`: Redirect to a named route

`away()`: External redirection (to another domain)
